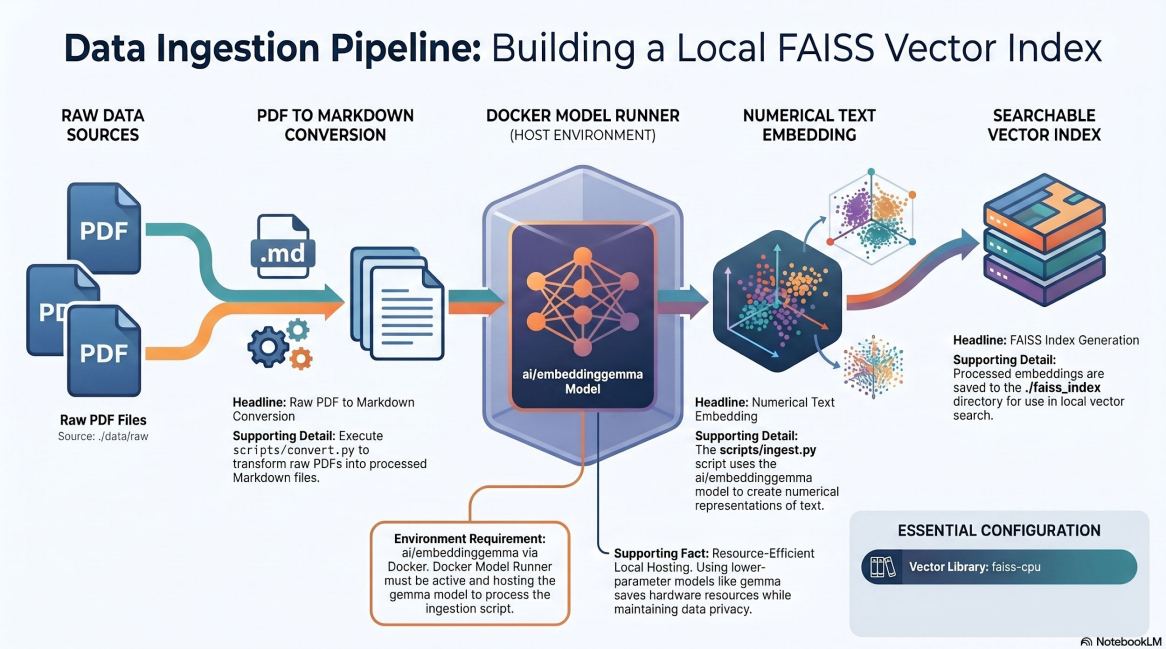
Data Ingestion and Index Creation

What is data ingestion and how does it work?
What is a **FAISS index** and what is it used for?
What does the data look like in the FAISS index?
How is the **ai/embeddinggemma** used to create the index?
What are the files **index.faiss** and **index.pkl** used for?

What is data ingestion?



NotebookLM

PDF → Text → **ai/embeddinggemma** → **numerical representation** → **searchable vector index**

Using **ai/embeddinggemma** to create Numeric Vectors

- Recall that **ai/embeddinggemma** is a model, but not an LLM model
- When we want to interact with **ai/embeddinggemma** it will be through and API call (demonstrated in the Docker Model Runner Set up Demo)
- The Docker application must be running to interact with LLMs and Models
- The “**input**” is that text that will be converted to a numeric vector

```
curl --location 'http://localhost:12434/engines/llama.cpp/v1/embeddings' \
--header 'Content-Type: application/json' \
--data '{
  "model": "ai/embeddinggemma",
  "input": "Your text to embed here"
}'
```

What does a FAISS index look like?


```
{"model":"ai/embeddinggemma",
"object":"list",
"usage":{"prompt_tokens":7,"total_tokens":7},
"data":[
{"embedding":[
0.05127084255218506,0.040495757013559 34,-0.04635112360119819 6,0
.03398002684116363 5,-0.013104524463415146,-0.009052153676748276
,-0.04446783661842346,-0.05297844856977463,-0.019724780693650246,-0.032239884138107 3,0.001111525227315724,-0.050529297441244125,0.045358631759881 97,0.036436580121517 18,0.001693782629445 1952
,0.0090109948068 8572,0.010691917501389 98,...
```

ingest.py

How does ingestion work?

/processed
MARKDOWN TEXT

onboarding
benefits
recognition
sick leave

Python script: **ingest.py**

ai/embeddinggemma model

Python Packages:
langchain_community
langchain_openai
langchain_text_splitters
langchain_core.documents

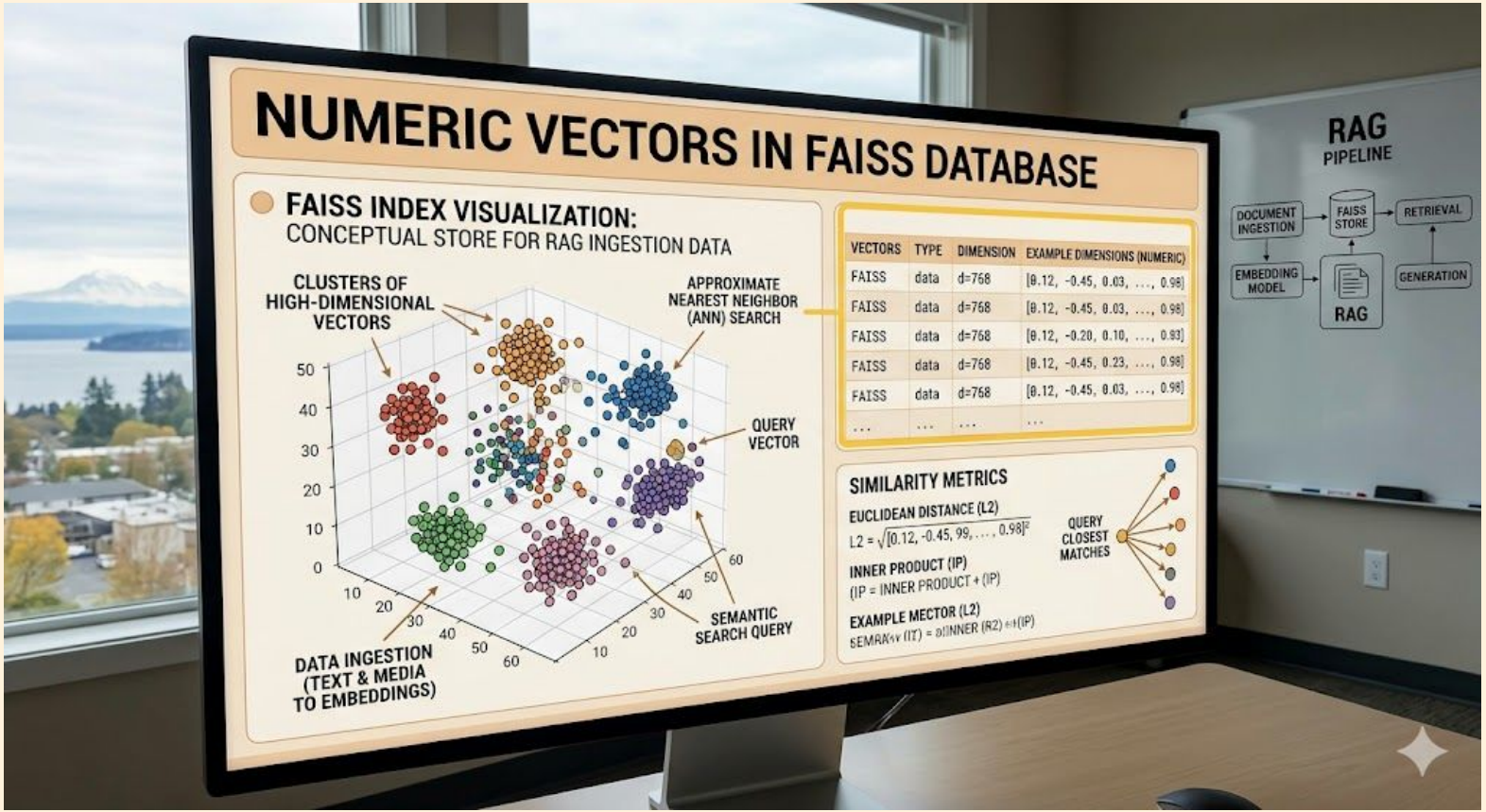
FAISS Index

Numeric Vector Store
`index.faiss`
`index.pkl`

What is a FAISS index and what is it used for?

- **FAISS** is an acronym for **F**acebook **AI** **S**imilarity **S**earch
  - open source library created by Meta
  - stores and search numeric vectors using metadata that connects to ingested text
  - numeric vectors can be retrieved and used to create context for LLM prompts
  - think of the index as a “database” that contains data in a form that allows semantic similarity detection
  - data is loaded using ingestion script and retrieved when a prompt is looking for searching for context

What does the data in the FAISS index look like?



Google Gemini

ingest.py

What do the Python packages and classes do?

```
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import
RecursiveCharacterTextSplitter
from langchain_core.documents import Document
```

Package	Class	Purpose
langchain_community.vectorstores	FAISS	Create vector index
langchain_openai	OpenAIEmbeddings	Convert text chunk into numeric vectors
langchain_text_splitters	RecursiveCharacterTextSplitter	Decides where to cut text in chunking
langchain_core	Document	Markdown text is read into Document

ingest.py

Chunking

```
# Chunk documents
# Large documents are split into smaller, overlapping chunks before embedding.
# This is necessary because embedding models have a token limit, and smaller chunks produce more focused, semantically meaningful vectors.
# RecursiveCharacterTextSplitter tries to split on natural boundaries
# (\n\n, \n, spaces) before falling back to hard character cuts.
# chunk_size=1000   : Maximum characters per chunk (~200-250 tokens)
# chunk_overlap=200 : Characters shared between adjacent chunks, preserving context at split boundaries so no sentence is cut mid-thought


splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = splitter.split_documents(docs)
```

ingest.py

Embedding

```
# Converts each text chunk into a high-dimensional numeric vector (embedding) that capture its semantic meaning. Chunks with similar meaning will have vectors that are close together in vector space, enabling similarity search.
# The model is served locally via Docker Model Runner on an OpenAI-compatible endpoint, so we use LangChain's OpenAIEmbeddings client pointed at localhost.
# The api_key value is a required parameter but is not validated by the local server.

embeddings = OpenAIEmbeddings(
    model="ai/embeddinggemma",
    base_url="http://localhost:12434/v1",  # Local Docker Model Runner endpoint
    api_key="not-needed"                   # No authentication required for local inference
)
```

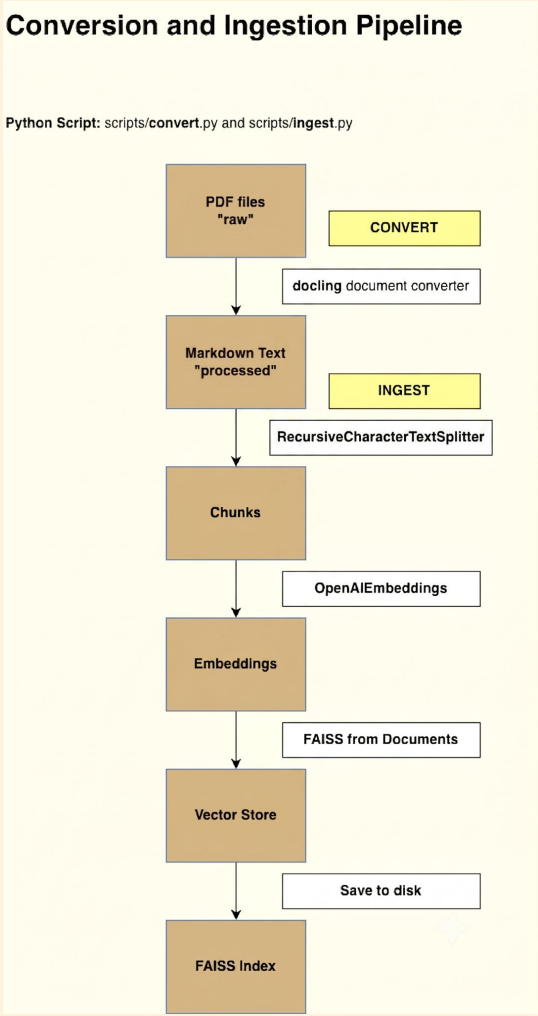
ingest.py

Chunking + Embedding = Vector Store

```
# Build the FAISS vector store from all embedded chunks.


vectorstore = FAISS.from_documents(
    documents=chunks,
    embedding=embeddings
)
# Save relative to project root
vectorstore.save_local(PROJECT_ROOT + "/faiss_index")
```

FAISS Index Creation Pipeline



Code Organization for Creation of FAISS index

```
rag_313/
|---- data/
|         /processed
|             …..md
|         /raw
|             …..pdf
|---- faiss_index/
|         index.faiss
|         index.pkl
|---- scripts/
|         convert.py
|         ingest.py
|---- app.py
```

Code Organization for Creation of FAISS index

`index.faiss`

- **store large datasets of numeric vectors**
- **quickly search large dataset of numeric vectors to find semantically similar items**

`index.pkl`

- **separate document metadata from numeric vectors**
- **stores document IDs and text content so that human readable text is available following search**

Next: DEMO Ingestion

- **Run scripts/test_embeddings.py**
- **Run ingestion script: scripts/ingest.py**
- **View output for index.faiss and index.pkl**

Ingestion

At this point, you're ready to run the code to create the FAISS index which is used to store and search for ingested user data.

Create FAISS Index

When you complete the actions demonstrated in this video, you will have created a FAISS index. There will be two files under the **faiss_index** directory named **index.faiss** and **index.pkl**.

Next Step:
Write the Application